A proposal for a type trait to detect narrowing conversions

ISO/IEC JTC1 SC22 WG21 D0870R6 2025-06-17

Project: Programming Language C++

Audience: Library Working Group, Library Evolution Working Group, Study Group 6 (Numerics)

Giuseppe D'Angelo, giuseppe.dangelo@kdab.com

Table of Contents

§ 1. Introduction

This paper proposes a new type trait for the C++ Standard Library, `is_convertible_without_narrowing`, to detect whether a type is implictly convertible to another type without going through a narrowing conversion.

§ 1.1 Tony Tables

Before

```
// list initialization forbids narrowing for e.g. int,
// but not for types "wrapping" int (like
// optional<int>, variant<int>, etc.):


template <typename T>
struct my_optional
{
    template <typename U = T>
    my_optional(U&& u)

    { ~~~ }
};

int some_int{3.14};              // ERROR
my_optional<int> optional_int{3.14}; // OK, but possibly not wanted
```

After

```
// make it possible to block narrowing for int,
// as well as anything "wrapping" int:


template <typename T>
struct my_optional
{
    template <typename U = T>
    my_optional(U&& u)
    requires std::is_convertible_without_narrowing_v<U, T>
    { ~~~ }
};

int some_int{3.14};              // ERROR
my_optional<int> optional_int{3.14}; // ERROR
```

Before

```
template <typename T>
class complex;

// Each floating point specialization of complex defines conversion
// constructors which are implicit if and only if they don't narrow

template <>
class complex<float>
{
    float re; float im;
public:
    explicit complex(const complex<double>& other);
    explicit complex(const complex<long double>& other);
};

template <>
class complex<double>
{
    double re; double im;
public:
    /* implicit */ complex(const complex<float>& other);
    explicit complex(const complex<long double>& other);
};

// repeat for long double, extended floating point [P1467R4], etc.
```

After

```
template <typename T>
class complex;

// Just one converting constructor template that uses explicit(b
// (it's pointless to also check for convertibility, floating po
// always convert to each other)

template <std::floating_point T>
class complex
{
    T re; T im;
public:
    template <std::floating_point U>
    explicit(!std::is_convertible_without_narrowing_v<U, T>)
    complex(const complex<U>& other);
};
```

Before

[variant.ctor], from [N4861]:

```
template<class T> constexpr variant(T&& t) noexcept(see
below);
```

After

```
template<class T> constexpr variant(T&& t) noexcept(see below);
```

Before

Let T_j be a type that is determined as follows: build an imaginary function `FUN(`T_i`)` for each alternative type T_i for which `$T_i$ x[] = {std::forward<T>(t)};` is well-formed for some invented variable `x`. […]

After

Let T_j be a type that is determined as follows: build an imaginary function `FUN(`T_i`)` for each alternative type T_i for which `is_convertible_without_narrowing_v<T&&, `T_i`>` is `true`. […]

Before

```
// simplified from QObject

template <typename Func1, typename Func2>
bool connect(QObject *sender, Func1 signal,
             QObject *receiver, Func2 slot)
{
    // Check that the signal is compatible with the slot.
    // This means:
    // * that the signal carries at least as many arguments
    //   than the slot;
    // * that each positional argument of the signal is convertible
    //   to the respective argument of the slot;
    // * etc.

    static_assert(detail::check_argument_count<Func1, Func2>,
                  "The slot requires more arguments than the signal provides.");

    static_assert(detail::check_convertible_arguments<Func1, Func2>,
                  "Signal and slot arguments are not compatible.");

    return detail::connect_impl(~~~);
}

// When the temperature changes, emits a signal carrying a double
BodyThermometer *thermometer = ~~~;

// Has a slot that shows an integer value on the screen
IntLabel *label = ~~~;

// OK, but likely a mistake: will silently truncate the double
// emitted with the signal. For instance, 37.0 and 37.99 would
// be both displayed as "37".
// Still, it compiles, because double is implicitly convertible to int...
connect(thermometer, &BodyThermometer::temperatureChanged,
        label, &IntLabel::setValue);
```

After

```
// simplified from QObject

template <typename Func1, typename Func2>
bool connect(QObject *sender, Func1 signal,
             QObject *receiver, Func2 slot)
{
    // Same checks, but refine the convertibility ch
    // carried by the signal needs a narrowing conve
    // in order to be converted to the respective ar

    static_assert(detail::check_argument_count<Func1
                  "The slot requires more arguments t

    static_assert(detail::check_convertible_without_
                  "Signal and slot arguments are not

    return detail::connect_impl(~~~);
}

BodyThermometer *thermometer = ~~~;

IntLabel *label = ~~~;

// ERROR: failing static_assert, would narrow
connect(thermometer, &BodyThermometer::temperatureCh
        label, &IntLabel::setValue);
```

§ 2. Changelog

- P0870R6
  - Added more details to the design decisions. No changes to the wording.
- P0870R5
  - Incorporated feedback after a LWG review; new wording has been provided.
  - Rebased on top of the latest draft.
  - Added a discussion about the deliberate choice of never considering the source to be a constant expression.
  - Added a reference to P2509R0.
  - Misc. reading improvements, typo fixes.
- P0870R4
  - Incorporated the feedback after a review of the paper on the lib-ext mailing list.
  - Changed the proposed name of the trait to `is_convertible_without_narrowing` following a poll on the mailing list; the semantics have been adapted.
  - Added Tony Tables with more examples.
  - Change the target audience to LEWG.
- P0870R3
  - Clarified that the implementation using an array won't work with certain datatypes (`void`, etc.).
  - Added a feature test macro.
  - Rebased the proposed wording on top of N4861.
  - Improved the proposed wording.
  - Editorial fixes: numbered the sections, fixed misspellings.
- P0870R2
  - Incorporated feedback from the LEWGI, EWGI, EWG sessions in Prague. In the LEWGI session this proposal was met favourably (2-10-2-1-0). EWGI and EWG didn't have any direct feedback (and also no particular concerns).
  - Elaborated on possible implementations.
  - Elaborated more on design decisions, especially regarding whether user-defined types should be handled, and whether this proposal would constrain core language evolution. Unlike R1, we now also include user-defined types in narrow conversion sequences (but like R1, we stick to [dcl.init.list] semantics, which are not customizable).
  - Updated reference to P0608R3.
  - Referenced other papers.
  - Incorporated the changes to [dcl.init.list] introduced by P1957R2.
  - Added SG6 to the target audience.
  - Improved spelling.
- P0870R1
  - Submitted for the Prague mailing, targeting LEWGI.

§ 3. Motivation and Scope

A *narrowing conversion* is formally defined by the C++ Standard in [dcl.init.list], with the intent of forbidding them from list-initializations.

It is useful in certain contexts to know whether a conversion between two types would qualify as a narrowing conversion. For instance, it may be useful to inhibit (via SFINAE) construction or conversion of "wrapper" types (like `std::variant` or `std::optional`) when a narrowing conversion is requested.

A use case the author has recently worked on was to prohibit narrowing conversions when establishing a connection in the Qt framework (see [Qt]), with the intent of making the type system more robust. Simplifying, a connection is established between a *signal* non-static member function of an object and a *slot* non-static member function of another object. The signal's and the slot's argument lists must have compatible arguments for the connection to be established: the signal must have at least as many arguments as the slot, and the each argument of the signal must be implictly convertible to the corresponding argument of the slot. In case of a mismatch, a compile-time error is generated. Bugs have been observed when connections were successfully established, but a narrowing conversion (and subsequent loss of data/precision) was happening. Detecting such narrowing conversions, and making the connection fail for such cases, would have prevented the bugs.

§ 4. Impact On The Standard

This proposal is a pure library extension.

It proposes changes to an existing header, `<type_traits>`, but it does not require changes to any standard classes or functions and it does not require changes to any of the standard requirement tables.

This proposal does not require any changes in the core language, and it has been implemented in standard C++.

This proposal does not depend on any other library extensions.

This proposal may help with the implementation of [P0608R3], the adopted resolution for [LEWG 227]. In particular, in [P0608R3]'s wording section, the "invented variable `x`" is used to detect a narrowing conversion. However, the same paragraph is also broader than the current proposal, in the sense that it special-cases boolean conversions [conv.bool], while the current proposal does not handle them in any special way. The part about boolean conversions has been anyhow removed by adoping [P1957R2].

The just mentioned [P1957R2], as a follow up of [P0608R3], made conversions from pointers to `bool` narrowing. It was adopted in the Prague meeting, therefore extending the definition of narrowing conversion in [dcl.init.list]. We do not see this as a problem: we aim to provide a trait to detect narrowing conversions exactly as specified by the Standard. Should the specification change, then the trait detection should also change accordingly; cf. the design decisions below.

This proposal overlaps with [P1818R1], a proposal that aims at introducing a distinction between narrowing and widening conversions. [P1818R1] is much more ambitious than this proposal; amongst other things, it aims at defining a `widening` trait. The wording of [P1818R1] is not complete, but we think that such a trait somehow overlaps with the semantics of the trait proposed here; it would ultimately depend on the exact definition of what constitutes a "widening conversion". Anyhow, the adoption of [P1818R1] is orthogonal to the present proposal.

[P1619R1] and [P1998R1] introduce functions (called respectively `can_convert` and `is_value_lossless_convertable`) that check whether a given value of an integer type can be represented by another integer type. This is in line with the spirit of detecting narrowing conversions, namely, preventing loss of information and/or preventing undefined behavior. While this proposal works on types, the functions examine specific values; we therefore think that the proposals are somehow orthogonal to the current proposal. Moreover, [P1619R1] and [P1998R1]'s functions only deal with integer types, while the narrowing definition in [dcl.init.list] (adopted by this proposal) also deals with floating point and enumeration types (and, following [P1957R2]'s adoption, also boolean and pointer types).

[P2509R0] proposes a type trait which complements the present proposal, by detecting if a given target type can represent all the values of a given source type. This is different from detecting a narrowing conversion as defined by the core language; for instance, converting `int` to `double` is a narrowing conversion even on common platforms where `double` can actually precisely represent any `int` value.

§ 5. Design Decisions

The most natural place to add the trait presented by this proposal is the already existing `<type_traits>` header, which collects most (if not all) of the type traits available from the Standard Library.

In the `<type_traits>` header the `is_convertible` type trait checks whether a type is implictly convertible to another type. Building upon this established name, the proposed name for the trait described by this proposal shall therefore be `is_convertible_without_narrowing`, with the corresponding `is_convertible_without_narrowing_v` type trait helper variable template.

§ 5.1 Should `is_convertible_v<From, To> == true` be a precondition of trying to instantiate `is_convertible_without_narrowing<From, To>`?

The author deems this unnecessary. Adding such a precondition would likely make the usage of the proposed trait more difficult and/or error prone.

As far as the author can see, the precondition is not going to dramatically simplify the specification for the proposed trait, or its implementation.

§ 5.2 Given a type `From` convertible to a type `To`, should `is_convertible_without_narrowing_v<From, To>` yield `false` only for the combinations of types listed in [dcl.init.list]? What about user-defined types?

The R1 revision of this paper was limiting the possibility for the trait to detect narrowing if and only if both `From` and `To` were one of the types listed in [dcl.init.list], which are all builtin types. Now, while we strongly want the type trait to match the language definition of narrowing conversion, there are some objections to simply detecting if we are in one of the those cases.

For instance, should a conversion from `const double` to `float` be considered narrowing or not? If we take the listing in [dcl.init.list] literally, then the answer would be "no" (that is, the trait would have to yield `true`), because `const double` is not in the list. However, this is not really useful, nor expected. If some code checks whether there is a conversion between two types and the conversion isn't narrowing, such code could erroneously infer that this is the case between `const double` and `float` and do the conversion. (This is in spite of the fact that list-initialization would not work, because *there is* a narrowing conversion in the conversion sequence).

To elaborate on this last sentence: the definition of list-initialization in [dcl.init.list] contains several cases where, if a narrowing conversion takes place, then the program is ill-formed. We believe that, more than detecting if a given conversion between two types *is* a narrowing conversion (as strictly per the definition), users want to detect if a conversion *has* a narrowing conversion, and thus would be forbidden if used in a list-initialization. As a consequence, we claim that `is_convertible_without_narrowing_v<const double, float>` should yield `false`.

This is not simply doable by stripping `From` and `To` of their *cv*-qualifiers (e.g. by applying `std::decay_t` on them), and then checking if the resulting types do not form a narrowing conversion. One must also consider all the other cases for conversions, which necessarily include user-defined types, as they may define implicit conversion

operators that can be used as part of a conversion sequence that also includes a narrowing conversion. As an example:

```
struct S { operator double() const; };
S s;
float f1 = s;     // OK
float f2 = { s }; // ERROR: narrowing
```

We therefore claim that `is_convertible_without_narrowing_v<S, float>` should yield `false`. A similar example is used in [P0608R3] and then [P1957R2]'s design. Slightly adapted:

```
struct Bad { operator char const*() &&; };
```

With the adoption of [P1957R2] in the Standard, we claim that `is_convertible_without_narrowing_v<Bad, bool>` should yield `false`. For completeness, `is_convertible_without_narrowing_v<Bad &, bool>` should yield `false` (there is no conversion), and `is_convertible_without_narrowing_v<Bad &&, bool>` should yield `false` as well (narrowing).

Note that, although user-defined datatypes are involved, the detection of whether there is a narrowing conversion sticks to the list of narrowing conversions in [dcl.list.init], which deals with fixed number of cases: between certain integer and floating point types; from unscoped enumeration types to a integer or a floating point type; and from pointer types to boolean. We are not proposing to allow any customization to these cases (see below).

§ 5.3 Should `is_convertible_without_narrowing<From, To>` yield `false` for boolean conversions [conv.bool]?

The author deems this to be out of scope for the proposed trait, which should have only the semantics defined by the language regarding narrowing conversion. Another trait should be therefore added to detect (implicit) boolean conversions, which, according to the current wording of [dcl.init.list], are not considered narrowing conversions. (Note that with the adoption of [P1957R2], conversions from pointer types to boolean are indeed considered narrowing.)

It is the author's opinion that, in general, implicit boolean conversions are undesirable for the same reasons that make narrowing conversions unwanted in certain contexts — in the sense that a conversion towards bool may discard important information. However, we recognize the importance of not imbuing a type trait with extra, not yet well-defined semantics, and therefore we are excluding boolean conversions from the present proposal.

§ 5.4 Should `is_convertible_without_narrowing<From, To>` use some other semantics than the narrowing defined in [dcl.init.list]?

Given that narrowing is formally defined in the core language, we are strongly against deviating from that definition. If anything, proposals aiming at marking as narrowing certain conversions between user defined types should also amend the core language definition; the type trait would still stick to that definition.

§ 5.5 Should specializations of `is_convertible_without_narrowing<From, To>` be allowed?

We believe that specializations of the trait should be forbidden. This is consistent with the other type traits defined in [meta] (cf. [meta.rqmts]), and with the fact that the current wording of [dcl.init.list] defining narrowing conversions does not extend to user-defined types.

We are aware of broader work happening in the area, such as [P1818R1]. That proposal introduces a keyword to mark widening conversions, but one may think of an alternative approach where users are expected to specialize `is_convertible_without_narrowing` in order to mark a conversion as non-narrowing. In any case, lifting the above limitation on specializations would always be possible.

§ 5.6 Does providing such a trait make changes to the definition of "narrowing conversion" harder to land?

This objection has been raised during the LEWGI meeting in Prague. The objection is sound, in the sense that a future possible change to the definition of narrowing conversion will impact user code using the trait.

On the other hand, during the Prague meeting, EWGI and EWG did not raise this concern regarding this proposal.

First and foremost, we would like to remark that the trait is already implementable — and has indeed already been implemented — using standard C++ (e.g. via `To{std::declval<From>()}`, cf. below). Any change to the definition of narrowing would therefore already be impacting user code. The impact would likely go beyond type traits / metaprogramming, since initialization itself would change semantics. An example of such a change affecting [dcl.init.list] comes from [P1957R2], which has been adopted in Prague. That adoption changed the semantics of code like `bool b = {ptr};` (making it ill-formed; was OK before).

In general, changes to the core language that affected type traits have already happened. For instance, between C++17 and C++20 the definition of "aggregate" in [dcl.init.aggr] has been changed, therefore changing the semantics of the `std::is_aggregate` type trait. Similarly, the definitions of "trivial class" and of "trivially copyable class" in [class.prop] have changed between C++14 and C++17; meaning that also the type traits `std::is_trivial` and `std::is_trivially_copy_constructible` have changed semantics.

We can therefore reasonably assume that the presence of a trait does not seem to preclude changes to the core language (or, if such an objection was raised when proposing these changes to the core language, it was also disregarded, as the changes did ultimately land).

Finally, it is the author's opinion that code that is using facilities to detect narrowing conversions would like to stick to the core language definition. If the definition changes, we foresee that the detection wants to change too, following the definition. Note that an ad-hoc solution (rather than a standardized trait) may fall out of sync with the core language definition, and thus start misdetecting some cases.

§ 5.7 Should there be a matching concept?

This point was raised during the LEWGI meeting in Prague, and it was deemed unnecessary at this point, because ultimately this proposal is just about a type trait. If it will be deemed useful, adding a concept could always be done at a later time, via another proposal.

§ 5.8 Bikeshedding: naming

The LEWGI review of the R3 revision of this paper discussed several options for the name of the trait.

In earlier versions we proposed the name `is_narrowing_convertible`; however, during the review of the paper, there was a general consensus that the majority of uses is going to be about detecting whether a conversion is possible *without* narrowing. As such, `is_narrowing_convertible` (which detects if a conversion does involve a narrowing conversion) would always need to be combined with `is_convertible` to achieve the desidered result, for instance like this:

```
~~~
// detect if From is convertible to To without a narrowing conversion
requires (is_convertible_v<From, To> && !is_narrowing_convertible_v<From, To>)
~~~
```

Another concern was raised about the very usage of "narrowing convertible", which is not currently a term used anywhere in the Standard.

Therefore, a poll was taken on the lib-ext mailing list, trying to find a better name, offering options for both a positive name (conversion sequence involves a narrowing conversion) or a negative one (conversion sequence does not involve a narrowing conversion). The proposed options were:

- `is_narrowing_convertible`
- `is_convertible_with_narrowing`
- `is_narrowing`
- `is_non_narrowing_convertible`
- `is_nonarrowing_convertible`
- `is_nonnarrowing_convertible`
- `is_convertible_without_narrowing`
- `is_convertible_no_narrowing`
- `is_non_narrowing`

The poll had very scarce participation, but `is_convertible_without_narrowing` emerged as the preferred option. The R4 revision of this paper renamed the trait, and, consequently, adapted the discussion regarding its semantics — that is, to detect types that are convertible through a conversion sequence that does *not* require a narrowing conversion. This required some rewording in a few places.

§ 5.9 The problem of detecting constant expressions as source

Many of the cases for narrowing conversions listed in [dcl.init.list] have an exception that states that a given conversion is **not** narrowing if *the source is a constant expression*, and the converted value satisfies certain criteria. For instance, consider this example:

```
int from = 42;
float to{from};   // ill-formed, narrowing
```

The initialization of `to` is ill-formed because of narrowing, although 42 is representable by a `float`. This is because the source is not a constant expression, so the relative provision in [dcl.init.list]/7.3 does not apply. Specifically, a lvalue-to-rvalue conversion is required for the initialization, and [expr.const]/5.9 makes the expression not a core constant expression.

It's however sufficient to change the snippet to this:

```
const int from = 42;    // add const here
float to{from};         // OK
```

and now there is no longer a narrowing conversion, because the source is a constant expression, and 42 can be represented exactly by both `float` and `int`.

A similar example is this one:

```
std::integral_constant<int, 42> from;    // not const
float to{from};                          // OK
```

Here the initialization of `to` is again well-formed. The source now is a constant expression, and 42 can be represented exactly. Although `from` is not "an object that is usable in constant expressions" ([expr.const]/4), there is nothing that disqualifies the expression from being a core constant expression.

(Again, specifically: the aforementioned lvalue-to-rvalue conversion does not apply here. The list-initialization of an object of type `float` ([dcl.init.general]/16.1) will resolve into direct-initialization ([dcl.init.list]/3.9); then, according to [dcl.init.general]/16.7, a user-defined conversion function is called and that "converts the initializer expression into the object being initialized". For the `std::integral_constant` class template, the conversion function (in the example, `from.operator int()`) is `constexpr`. Then, the prvalue `int` obtained as a result is converted to `float` using a floating-integral conversion ([conv.fpint]/2). In conclusion, this means that the source is a constant expression, and there is *not* a narrowing conversion in the initialization of `to`.)

It may appear that these considerations need to affect the design of the trait: should `is_convertible_without_narrowing_v<int, float>` yield `true` or `false`? What about `is_convertible_without_narrowing_v<const int, float>`? `is_convertible_without_narrowing_v<std::integral_constant<int, 42>, float>`?

The answer to these questions lies in the fact that the "exceptions" to the narrowing conversion rules are all based on the source being a constant expression, *and that is a quality that is not reflected in any way in the type system*.

For instance, a `const int` variable may or may not be (usable in) a constant expression depending on how and where it gets initialized ([expr.const]/2, 3, 4). Converting such a variable to a `float` may or may not be a narrowing conversion. We do not have any means to know if it's the case from the *types* alone — not to mention the values.

For this reason, a type trait like the one that we are proposing will never be able to provide a detection that is *complete*, but only a *correct* one (no false positives). This does not undermine the usefulness of the trait, which is first and foremost to avoid conversions that may result in accidental loss of data.

Therefore, as a design decision, we are going to assume that the source expression is **never** a constant expression, and have the trait give the corresponding answer. Practically speaking, we believe this matches the main use case of this trait: constraining functions from being called with parameters of types that may narrow when converted to some other type. These parameters are usually not going to be (usable in) constant expressions anyways:

```
template <typename T>
struct my_optional
{
    template <typename U = T>
        requires std::is_convertible_without_narrowing_v<U, T>
    my_optional(U&& u)    // `u` not usable in constant expression
      : data{std::forward<U>(u)}, loaded{true} {}
};
```

(A related proposal in this area is P1045R1 ("`constexpr` Function Parameters"). The possibility of such parameters would have implications for the trait. However, work on that proposal seems to have stalled.)

As for `is_convertible_without_narrowing_v<std::integral_constant<int, 42>, float>`, we believe the trait should return **`true`**. Although an object of type `std::integral_constant<int, 42>` is not necessarily usable in a constexpr context, the *conversion* of such an object towards e.g. `float` is a constant expression:

```
std::integral_constant<int, 42> value; // not constexpr
constexpr float f{value}; // OK
```

Therefore, the trait should correctly determine that we are in a situation where where the conversion is not narrowing, and yield `true`.

It's important to note that this behavior matches the intended behavior of `std::variant`'s converting constructor, as discussed in P3146R2 ("Clarifying std::variant converting construction"), which has been approved by LEWG.

§ 5.10 Usage of `std::declval` in the wording

The proposed wording is expressed in terms of `std::is_convertible`, by stating that "the conversion, as defined by `is_convertible`, does not require a narrowing conversion.

It has been pointed out that `is_convertible` specification, at the moment, is formulated in terms of a call to `std::declval()`. This has an impact on the trait, as `std::declval` is not a constexpr function.

The impact has been discussed in the previous paragraph: the source value is never considered to be a constant expression.

§ 6 Implementation Experience

The proposed type trait has already been implemented in various codebases using ad-hoc solutions (depending on the quality of the compiler, etc.), using standard C++ and without the need of any special compiler hooks.

One possible implementation is to exhaustively check whether a conversion between two types fall in one of the narrowing cases listed in [dcl.list.init]. This particular implementation has been done in [Qt 5's `connect()`] (see this proposal's motivation about why this detection has been added to Qt in the first place); it has obviously maintenance and complexity shortcomings. The most important part is:

```
template<typename From, typename To, typename Enable = void>
struct AreArgumentsNarrowedBase : std::false_type
{
};

template<typename From, typename To>
struct AreArgumentsNarrowedBase<From, To, typename std::enable_if<sizeof(From) && sizeof(To)>::type>
    : std::integral_constant<bool,
          (std::is_floating_point<From>::value && std::is_integral<To>::value) ||
          (std::is_floating_point<From>::value && std::is_floating_point<To>::value && sizeof(From) > sizeof(To)) ||
          ((std::is_integral<From>::value || std::is_enum<From>::value) && std::is_floating_point<To>::value) ||
          (std::is_integral<From>::value && std::is_integral<To>::value
           && (sizeof(From) > sizeof(To)
               || (std::is_signed<From>::value ? !std::is_signed<To>::value
                   : (std::is_signed<To>::value && sizeof(From) == sizeof(To))))) ||
          (std::is_enum<From>::value && std::is_integral<To>::value
           && (sizeof(From) > sizeof(To)
               || (IsEnumUnderlyingTypeSigned<From>::value ? !std::is_signed<To>::value
                   : (std::is_signed<To>::value && sizeof(From) == sizeof(To)))))
          >
{
};
```

(Historical note: at the time this implementation was done, the codebase could only use C++11 features, and some compilers were unable to cope with the SFINAE solution presented below.).

The Qt implementation strictly checks for the one of narrowing cases listed in [dcl.init.list], which is not the aim of the current proposal (cf. design decisions above). Also, the snippet does not yet implement the change introduced by [P1957R2], showing an inherent limitation to such an "exhaustive" solution: it may fall out of sync with the core language.

Another, much simpler implementation is to rely on SFINAE around an expression such as `To{std::declval<From>()}`. However, this form is also error-prone: it may accidentally select aggregate initialization for `To`, or a constructor taking a `std::initializer_list`, and so on. If for any reason these make the expression ill-formed, then the trait is going to report that there is a narrowing conversion between the types, even when there actually isn't one (or vice-versa, depending on how one builds the trait).

[P0608R3] uses a clever workaround to avoid falling into these traps: declaring an array, that is, building an expression like `To t[] = { std::declval<From>() };`. This solution is now used in the Standard (in [variant.ctor] and [variant.assign]), amongst other things, in order to detect and prevent a narrowing conversion. The trait that we are proposing could be used to replace the ad-hoc solution, and clarify the intent of it.

A possible implementation that uses the just mentioned workaround is the following one (courtesy of Zhihao Yuan; received via private communication, adapted):

```
template<class From, class To>
concept is_convertible_without_narrowing = std::is_convertible_v<From, To> &&
  requires (From&& x) {
    { std::type_identity_t<To[]>{std::forward<From>(x)} } -> std::same_as<To[1]>;
  };
```

(A C++17-compatible version of the above implementation has been implemented in Qt 6.)

While an array declaration can be used as the main part of detection, a complete implementation also needs handling of the "corner cases". Types such as *cv* `void`, reference types, function types, arrays of unknown bounds, etc., require special care because they cannot be used to declare the array of `To`, and this may give wrong results. Note that we propose that it is legal to specialize `is_convertible_without_narrowing` with these types, following the precedent set by `is_convertible`.

This handling of "corner cases" is not much different anyhow from the practical implementations of other type traits: for instance, in both [libstdc++] and [libc++] implementations of `is_convertible` there is code that deals with some types in a special way.

In conclusion, it is the author's opinion that `is_convertible_without_narrowing` can be implemented in standard C++ without placing an unreasonable burden on implementors.

Author's note: given the amount of subtleties involved, we claim that it is extremely likely that non-experts would go for a "naïve", wrong solution (e.g. `To{std::declval<From>()}`). This further underlines the necessity of providing detection of narrowing conversions in the Standard Library, rather than having users reinventing it with ad-hoc solutions.

§ 7. Technical Specifications

§ 7.1. Feature testing macro

We propose the usage of the `__cpp_lib_is_convertible_without_narrowing` macro, following the pre-existing nomenclature for the other traits in `<type_traits>`.

§ 7.2. Proposed wording

All changes are relative to [N4928].

In [meta.type.synop], add to the header synopsis:

```
  template <class From, class To> struct is_convertible;
  template <class From, class To> struct is_nothrow_convertible;
+ template <class From, class To> struct is_convertible_without_narrowing;
```

```
  template <class From, class To>
    constexpr bool is_convertible_v = is_convertible<From, To>::value;
  template <class From, class To>
    constexpr bool is_nothrow_convertible_v = is_nothrow_convertible<From, To>::value;
+ template <class From, class To>
+   constexpr bool is_convertible_without_narrowing_v = is_convertible_without_narrowing<From, To>::value;
```

In [meta.rel], insert a new row to the *Type relationship predicates* table (Table 50) after the row for `is_nothrow_convertible`, with these contents in each column:

> *Template* `template <class From, class To> struct is_convertible_without_narrowing;`
>
> *Condition* `is_convertible_v<From, To>` is `true` and the conversion, as defined by `is_convertible`, does not require a narrowing conversion ([dcl.init.list]).
>
> *Comments* `From` and `To` shall be complete types, *cv* `void`, or arrays of unknown bound.

In [version.syn], add to the list of feature testing macros in paragraph 2:

```
#define __cpp_lib_is_convertible_without_narrowing   TBD // also in <type_traits>
```

The actual value is left to be determined (`TBD`); it should be replaced by `yyyymmL`, matching the year and month of adoption of this proposal.

§ Appendix

§ A. Acknowledgements

Thanks to KDAB for supporting this work.

Thanks to Marc Mutz for his feedback and championing this paper in Prague; to Zhihao Yuan for the discussion and some sample code; to Walter E. Brown and André Somers for their feedback.

All remaining errors are ours and ours only.

§ B. References

[Qt] Qt version 5.15, *QT_NO_NARROWING_CONVERSIONS_IN_CONNECT*, QObject class documentation

[Qt `connect()`] Qt version 5.15, *traits for detecting narrowing conversions*

[Qt `connect()`] Qt version 6.4.2, *traits for detecting narrowing conversions*

[P0608R3] Zhihao Yuan, *A sane variant converting constructor (LEWG 227)*

[LEWG 227] Zhihao Yuan, *Bug 227 - variant converting constructor allows unintended conversions*

[P1957R2] Zhihao Yuan, *Converting from `T*` to `bool` should be considered narrowing (re: US 212)*

[P1818R1] Lawrence Crowl, *Narrowing and Widening Conversions*

[P1619R1] Lisa Lippincott, *Functions for Testing Boundary Conditions on Integer Operations*

[P1998R1] Ryan McDougall, *Simple Facility for Lossless Integer Conversion*

[libstdc++] libstdc++, *implementation of `is_convertible`*

[libc++] libc++, *implementation of `is_convertible`*

[N4928] Thomas Koeppe, *Working Draft, Standard for Programming Language C++*

[P1467R4] David Olsen, Michał Dominiak, *Extended floating-point types and standard names*

[P2509R0] Giuseppe D'Angelo, *A proposal for a type trait to detect value-preserving conversions*

[P1045R1] David Stone, *constexpr Function Parameters*

[P3146R2] Giuseppe D'Angelo, *Clarifying std::variant converting construction*